

[HOME](#) [TOP](#) [CATALOG](#) [CONTESTS](#) [GYM](#) [PROBLEMSET](#) [GROUPS](#) [RATING](#) [EDU](#) [API](#) [CALENDAR](#) [HELP](#)
[AWOO](#) [BLOG](#) [TEAMS](#) [SUBMISSIONS](#) [GROUPS](#) [CONTESTS](#) [PROBLEMSETTING](#) [STREAMS](#)

awoo's blog

Educational Codeforces Round 92 Editorial

By [awoo](#), [history](#), 6 years ago, translation, 

1389A - LCM Problem

Idea: [BledDest](#)[Tutorial](#)[Solution \(BledDest\)](#)

1389B - Array Walk

Idea: [Roms](#)[Tutorial](#)

1389B - Array Walk

Notice that your final position is determined by the number of moves to the left you make. Let there be exactly t moves to the left, that leaves us with $k - t$ moves to the right. However, let's interpret this the other way. You have t pairs of moves (right, left) to insert somewhere inside the sequence of $k - 2t$ moves to the right.

Easy to see that all the positions from 1 to $k - 2t + 1$ will always be visited. And the extra pairs can also increase the score by visiting some positions $(i + 1, i)$ for some i from 1 to $k - 2t + 1$.

Notice that it's always optimal to choose exactly the same i for all the pairs (right, left). And that i should be such that $a_{i+1} + a_i$ is maximum possible.

You can implement this idea in a straightforward manner: iterate over t and calculate the sum of values from 1 to $k - 2t + 1$ and the maximum value of $a_{i+1} + a_i$ over i from 1 to $k - 2t + 1$.

That will lead to a $O(zn)$ solution per testcase.

You can optimize it to $O(n)$ with prefix sums or with some clever order to iterate over t . It's also possible to iterate over the final position and restore the number of left moves required to achieve it.

Overall complexity: $O(zn)$ or $O(n)$ per testcase.

[Solution 1 \(pikmike\)](#)

```
for _ in range(int(input())):
    n, k, z = map(int, input().split())
    a = [int(x) for x in input().split()]
    ans = 0
    s = 0
    mx = 0
    for t in range(z + 1):
        pos = k - 2 * t
```

→ Pay attention

Before contest

[Codeforces Round 1109 \(Div. 3\)](#)

36:09:29

[Register now »](#)

→ jiashenli40



Rating: 434

Contribution: 0

- [Settings](#)
- [Blog](#)
- [Teams](#)
- [Submissions](#)
- [Favourites](#)
- [Talks](#)
- [Contests](#)



jiashenli40

→ Top rated

#	User	Rating
1	jiangly	3728
2	VivaciousAubergine	3647
3	Kevin114514	3625
4	maroonrk	3555
5	strapple	3515
6	tourist	3439
7	Um_nik	3376
7	turmax	3376
9	Radewoosh	3367
10	heuristica	3322

[Countries](#) | [Cities](#) | [Organizations](#)
[View all →](#)

→ Top contributors

#	User	Contrib.
1	Qingyu	163
2	Um_nik	145
3	adamant	142
4	Dominator069	136
5	chromate00	133
5	DNR	133
5	maroonrk	133
8	cry	132
8	maspy	132
10	Proof_by_QED	131

[View all →](#)

```
if pos < 0:
    continue

mx = 0
s = 0
for i in range(pos + 1):
    if i < n - 1:
        mx = max(mx, a[i] + a[i + 1])
    s += a[i]
ans = max(ans, s + mx * t)

print(ans)
```

[Solution 2 \(pikmike\)](#)

1389C - Good String

Idea: **BledDest**

[Tutorial](#)

[Solution \(Ne0n25\)](#)

1389D - Segment Intersections

Idea: **adedalic**

[Tutorial](#)

[Solution \(adedalic\)](#)

1389E - Calendar Ambiguity

Idea: **BledDest**

[Tutorial](#)

[Solution \(pikmike\)](#)

1389F - Bicolored Segments

Idea: **Roms**

[Tutorial](#)

[Solution 1 \(Ne0n25\)](#)

[Solution 2 \(Ne0n25\)](#)

1389G - Directing Edges

Idea: **BledDest**

[Tutorial](#)

[Solution \(BledDest\)](#)

Tutorial of Educational Codeforces Round 92 (Rated for Div. 2)

+127

[awoo](#)

6 years ago

107

Comments (67)

☐ Show archived | [Write comment?](#)



RisingWizard

6 years ago, [hide](#) # |

Super Fast!!

→ [Reply](#)

→ [Find user](#)

Handle:

[Find](#)

→ [Recent actions](#)

[atcoder_official](#) → [AtCoder Regular Contest-- 224 Announcement](#)

[_baozii](#) → [Spectral::Cup 2026 Round 3 \(Codeforces Round 1110, Div. 1 + Div. 2\)](#)

[ezraft](#) → [Codeforces Round 1108 \(Div. 2\)](#)

[Ahmad_OS](#) → [How do you debug?](#)

[DNR](#) → [AI resolves yet another major conjecture in math](#)

[Omar](#) → [Codeforces Round 1108 \(Div. 2\) Editorial](#)

[itz_pabloo](#) → [Codeforces Round 1109 \(Div. 3\)](#)

[abhi_yo](#) → [Grid problem](#)

[MikeMirzayanov](#) → [Rule Restricting the Use of AI \[revision 2024-09-14\]](#)

[VanLam](#) → [Solution Contest đầu vào CLB IT PTIT 2025](#)

[dino_merlin](#) → [Finally specialist again after 3 long, horrible, exruciating years](#)

[accord](#) → [Competitive Programming in the Era of AI](#)

[BledDest](#) → [Educational Codeforces Round 192 Editorial](#)

[makeinverse](#) → [CP Insights — Beautiful Codeforces Cards for GitHub READMEs](#)

[AmShZ](#) → [Repovive Premier Round 6](#)

[mambo](#) → [how are we not all suicidal](#)

[pushedx](#) → [How do I set my home country on Codeforces?](#)

[ShulkerBox](#) → [Smallest CF contest ever...](#)

[brave_art_37](#) → [Stuck in CP 1.5 years during college. Pls help](#)

[MidnightCodeCup](#) → [Midnight Code Cup Finals 2026: Problems, Winners and More](#)

[sarvan_2187](#) → [Hello, A Bit about Me!](#)

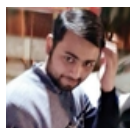
[MikeMirzayanov](#) → [Rule about third-party code is changing](#)

[NetSpeed1](#) → [Codeforces Global Round 28 Editorial](#)

[ABalobanov](#) → [Codeforces Round 1070 \(Div.2\) Editorial](#)

[atcoder_official](#) → [AtCoder Beginner Contest 466 Announcement](#)

[Detailed →](#)



iabhishek15

6 years ago, [hide](#) # ^ | ☆

← Rev. 2 ▲ +45 ▼

Day by day I feel like nobody is getting the humor anymore they are just downvoting every comment.

→ [Reply](#)

OmarKimo

6 years ago, [hide](#) # | ☆

← Rev. 2 ▲ 0 ▼

In the Tutorial of Problem E: I think you mean $yd + x$ instead of $xd + y$ as the index of the x -day of the y -month in the year.

→ [Reply](#)

Rahul_kumar

6 years ago, [hide](#) # ^ | ☆

▲ 0 ▼

$xd+y$ means index y -th day of x -th month which is corresponding pair of x -th day of y -th month whose index is $yd+x$.

→ [Reply](#)

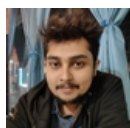
terencehill

6 years ago, [hide](#) # | ☆

▲ 0 ▼

Can someone tell me why my solution for D fails? I simply decide whether to gain intersection by joining two new segments or expanding others which already cover themselves totally. Counter-examples are especially appreciated.

Here's the submission: <https://codeforces.com/contest/1389/submission/88410212>.

→ [Reply](#)

adnan_toky

6 years ago, [hide](#) # ^ | ☆

▲ 0 ▼

The test, your solution fails when

```
n = 10, k = 297
l1 = 741, r1 = 741
l2 = 20, r2 = 770
```

Try to fix it.

→ [Reply](#)

terencehill

6 years ago, [hide](#) # ^ | ☆

▲ +3 ▼

Got my error, thank you!

→ [Reply](#)

grapo_Oranges

6 years ago, [hide](#) # ^ | ☆

▲ 0 ▼

Is there any way to solve D in $O(1)$.. maybe little case work (*chukles*), but I guess that could be done!

→ [Reply](#)

samill

5 years ago, [hide](#) # ^ | ☆

▲ 0 ▼

Yes. Use some greedy approach to choose how many segments need to be invested. This is my accepted approach <https://codeforces.com/contest/1389/submission/106042967>

→ [Reply](#)

grapo_Oranges

5 years ago, [hide](#) # ^ | ☆

▲ 0 ▼

nice, I'll try to solve it too in $O(1)$

→ [Reply](#)

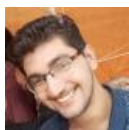
abhiaps

6 years ago, [hide](#) # | ☆

▲ 0 ▼

Waiting for Div 3 :) After so many difficult rounds to increase rating :(

→ [Reply](#)



ValarDohaeris_

6 years ago, [hide](#) <#> | ☆

▲ 0 ▼

What is **Event Processing** I see it here and there but never find some concise tutorial on it.

→ [Reply](#)

spookywooky

6 years ago, [hide](#) <#> ^ | ☆

▲ +8 ▼

<https://www.geeksforgeeks.org/maximum-number-of-overlapping-intervals/>

In that algo the beginning and ending of intervals is treated as "events" where the count of them is incremented or decremented.

→ [Reply](#)

ValarDohaeris_

6 years ago, [hide](#) <#> ^ | ☆

▲ 0 ▼

Thanks man!

→ [Reply](#)6 years ago, [hide](#) <#> | ☆

▲ 0 ▼

I think we can just go through 45 combinations for C instead of the 100 given in the tutorial.

TLDR:

0101010 has both a 101010 and a 010101 type strings in it.

DETAILED:

Let us say the string we get after removing all other numbers but two be some 011100001001010.....

Then for the above we want it to reduce to the form : 01010101... Note, that if the reduced string ends with 1 then the answer is: $\text{size}(\text{original string}) - \text{size}(\text{reduced string})$. Else, the answer is: $\text{size}(\text{original string}) - \{\text{size}(\text{reduced string}) - 1\}$.

→ [Reply](#)

munditva

6 years ago, [hide](#) <#> | ☆

▲ +3 ▼

Can someone explain the dynamic programming approach to problem E?

→ [Reply](#)

munditva



jgh99

6 years ago, [hide](#) <#> ^ | ☆

▲ 0 ▼

Problem E doesn't use dynamic programming. It's just math and number theory.

→ [Reply](#)

huayucaiji

6 years ago, [hide](#) <#> | ☆

▲ +5 ▼

Cool contest! Though I've lost some points :(

→ [Reply](#)

harry_122

6 years ago, [hide](#) <#> | ☆

← Rev. 3 ▲ 0 ▼

.

→ [Reply](#)

6 years ago, [hide](#) # [^](#) | [☆](#)

▲ 0 ▼

Firstly.

What does these numbers mean? Test containing 4 numbers, but u give 6. It's not 1 test and not 2.



saprykin

Secondly

X can't be negative.

In both pairs $\{l, r\} \mid l \leq r$, so $\max(r_1, r_2) \geq \max(l_1, l_2) \Rightarrow \max(r_1, r_2) \geq \min(l_1, l_2)$

I hope you understand

→ [Reply](#)



harry_122

6 years ago, [hide](#) # [^](#) | [☆](#)

← Rev. 2

▲ 0 ▼

.

→ [Reply](#)

6 years ago, [hide](#) # [^](#) | [☆](#)

▲ 0 ▼

Sample are incorrect and occasionally it's work (but it shouldn't)



saprykin

$$1 \ (1 \leq l_1 \leq r_1 \leq 10^9) -$$

$$2 \ (1 \leq l_2 \leq r_2 \leq 10^9) -$$

→ [Reply](#)



a4aakash

6 years ago, [hide](#) # | [☆](#)

← Rev. 4

▲ 0 ▼

Why this code is giving TLE? <https://ideone.com/wXdX3A>

→ [Reply](#)



saprykin

6 years ago, [hide](#) # [^](#) | [☆](#)

▲ 0 ▼

Lets calculate $O(10 * 10 * n + 10 * n) = O(100 * n)$ $n = 2 * 10^5$. We get $O(2 * 10^7)$. Its about 10 secs on Python and about 0.2-0.3 secs on C++ (I have solve with the same asymptotics) Friendly advice. Use C++ in real competitions.

→ [Reply](#)



a4aakash

6 years ago, [hide](#) # [^](#) | [☆](#)

▲ 0 ▼

Same code gives AC in c++ but tle in python :) btw thank you.

→ [Reply](#)



saprykin

6 years ago, [hide](#) # [^](#) | [☆](#)

▲ 0 ▼

C++ ~100 times faster than python

→ [Reply](#)



jolt.tks

6 years ago, [hide](#) # [^](#) | [☆](#)

▲ 0 ▼

always use fast I/O in Python. normal I/O in python takes a lot of time.
<https://www.geeksforgeeks.org/python-input-methods-competitive-programming/>

→ [Reply](#)



6 years ago, [hide](#) # | ☆

← Rev. 10 ▲ 0 ▼

Hey @adedalic I have one doubt in D, You have said,

In the **case of** non-intersecting $[l1, l2]$ and $[r1, r2]$, we should at first "invest" some number **of** steps **in** each pair to make them intersect. **So let's** iterate over the number of segments to "invest" $cntInv$. We should make $cntInv \cdot (\max(l1, l2) - \min(r1, r2))$ steps to make segments touch. Now, $cntInv$ segments touch so we can use almost the same formulas for them as in the previous case.

But consider **case where** input **is**,

5 16

1 4

8 12

In **this case** should we invest time **in** joining all pairs **or** just invest time **in** joining one pair

and complete operations **using** only that pair.

So steps **for** joining = 4

Steps to make both equal size = 11, remaining $k = 5$.

So instead **of** joining all pairs just invest **in** first joined pair.

Remaining steps = $5 \cdot 2 = 10$.

Ans = $4 + 11 + 10$.

Why should we invest **in** joining all pairs ?

Am I missing something ?

Community, **Please** help !!!

EDIT: Now I get it, u have used a for loop (i to n), that checks by joining first i pairs and compares it with minimum each time. Thanks.

→ [Reply](#)

↑
1
↓

6 years ago, [hide](#) # ^ | ☆

▲ 0 ▼

Just iterate over n and find minimum value.



zoro_2278

```
ans = INF
```

```
for i in :n
```

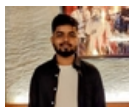
```
    temp = cost of getting K intersection with i  
    segments joined.
```

```
    ans = min(ans, temp)
```

→ [Reply](#)

6 years ago, [hide](#) # | ☆

▲ 0 ▼



YashDwivedi

I wrote a recursive solution using memoization where I checked for each I it's $i-1$ and $i+1$ and updated my answer with maximum sum Does my solution not violate the condition given in the question :. " you can't perform two or more moves to the left in a row" Can anyone please explain this that how my function goes for 2 or more left moves in a row if $z \geq 2$ and does not violate this and gets accepted

__88418038

→ [Reply](#)



sup_bro

6 years ago, [hide](#) # | ☆

▲ +42 ▼

Could anyone please explain the dp with segment tree approach for problem F?

→ [Reply](#)

6 years ago, [hide](#) # ^ | ☆

▲ +13 ▼

Here's how I understand it (I did not solve in contest), feel free to correct me if I'm wrong or there is an easier way to think about this.

First, let us sort all segments in increasing order of their right endpoints in the list *segs*. Split the segments based on type and sort them in the same fashion in the lists *s[t]*. Let $dp[i][j][t]$ be the answer assuming that the problem is restricted to the first *j* segments in *segs*, along with the condition that we can only select the first *i* segments in *s[t]*, and that we have used segment *i* in *s[t]* in our solution.

Now, to transition to *segs[j + 1]*, assuming *segs[j + 1]* is of type 1 and is the *k + 1*'th segment in *s[1]*, we have the following:

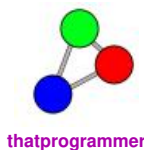
- $dp[i][j + 1][2] = dp[i][j][2] + 1$ if there is no touching between segment *k + 1* of *s[1]* and segment *i* of *s[2]*, and $dp[i][j + 1][2] = dp[i][j][2]$ otherwise.
- $dp[i][j + 1][1] = dp[i][j][1]$ for all $i \leq k$, and $dp[k + 1][j + 1][1]$ is the max of $dp[i][j + 1][2]$ for all segments *i* in *s[2]* that do not touch with segment *k + 1* in *s[1]*.

In other words, the dp table is largely unchanged, except that we try to add *s[1][k + 1]* to everything that we can, which must be optimal. The transition for where the new segment is of type 2 is very similar. Our answer is the maximum over the whole dp table.

Now, notice that for any new segment, the segments that do not touch it must be a prefix of the sorted segment list. Thus, these transitions are exactly range add modifications and maximum queries on two segment trees. We can easily find what range to add by using a binary search. Finally, we can eliminate the 2nd dimension with *j* just by directly modifying the segment trees.

See my [submission](#) for implementation details.

→ [Reply](#)

6 years ago, [hide](#) # ^ | ☆

▲ +8 ▼

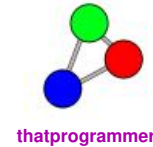
So, for implementation purpose, we can ignore the 2nd dimension of the dp state you mentioned, right? So our dp state will be:

$dp(i, j)$: maximum segments if we have only considered till the *i*th segment of type *j* and this one is included. And we traverse the segments on the sorted order of their right endpoints.

→ [Reply](#)

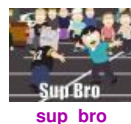
6 years ago, [hide](#) # ^ | ☆

▲ +8 ▼



Yes, exactly. When I was trying to figure the solution out based on other people's code, I found the 3 dimensions to be easier (as a solution that I could actually come up with), but you have it exactly right.

→ [Reply](#)

6 years ago, [hide](#) # ^ | ☆

▲ 0 ▼

Okay, sounds good. Thanks a lot.

→ [Reply](#)





Tathagat_shah

6 years ago, [hide](#) <#> | ☆

▲ +5 ▼

O(1) solution for D [88447760](#)→ [Reply](#)

speshuric

6 years ago, [hide](#) <#> ^ | ☆

▲ 0 ▼

+1 [88435747](#) in Kotlin→ [Reply](#)

confusedman

6 years ago, [hide](#) <#> | ☆

▲ 0 ▼

Could somebody please explain the dp approach to B?

→ [Reply](#)6 years ago, [hide](#) <#> ^ | ☆

← Rev. 2 ▲ 0 ▼

Dp approach of B will be like that we have two choices at every index either we can move left or right, we will explore the moves and the maximum between these two options will be considered. base cases are:



anurag122898

1 . when we don't have any moves left ($k==0$), in this case, we will return the current index value.

2 . when the last move taken were left or we don't have any left moves remained or we are at the starting index of the array, then we have only one option to move right at this index.

3.else we have two option to move right and left, and take the maximum among those to the answer. you can check my solution for more clarity, [88714266](#).

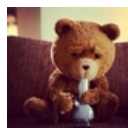
→ [Reply](#)

VaibhaveS

5 years ago, [hide](#) <#> ^ | ☆

▲ 0 ▼

How did you decide the states to cache? because here index,prev etc also change in every state...

→ [Reply](#)

Darshann

5 years ago, [hide](#) <#> ^ | ☆

← Rev. 2 ▲ 0 ▼

Even I had the same query [VaibhaveS](#). Could you explain it if you've understood it?

→ [Reply](#)

ashutosh_122

6 years ago, [hide](#) <#> | ☆

▲ 0 ▼

Can anyone help me with my code for Problem C ? Getting a Wrong Answer.

```
def func():
    cnst = "01233456789"

    test = input()

    ans = 1000000000

    for i in cnst:
        for j in cnst:
            c1 = 0
            c2 = 0
            for k in range(len(test)):
                if (i != j and len(test) % 2
```





```

-- 1 and k == len(test) - 1).

        continue
    elif (k%2 == 0):
        if (test[k] == i):
            c1 += 1
        else:
            if (test[k] == j):
                c2 += 1

    c111 = c1 + c2
    ans = min (len(test) - c111, ans)

    print (ans)

```

```

t = int(input())
for i in range(t):
    func()

```

→ [Reply](#)

6 years ago, [hide](#) # ^ | ☆

▲ 0 ▼



pratiksbbhati

Your code is checking if i is at even and j is at odd positions, which is wrong. You need to find the longest SUBSEQUENCE of alternating i and j of even length. INPUT: 1 2025 YOUR OUTPUT: 1 CORRECT OUTPUT : 2

→ [Reply](#)

6 years ago, [hide](#) # | ☆

← Rev. 5

▲ 0 ▼



strategy

Same approach as Editorial still can't be accepted in python3 due to TLE on test 2 , what is this?

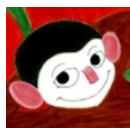
1389C-Good String [88530443](#)

→ [Reply](#)

6 years ago, [hide](#) # ^ | ☆

← Rev. 2

▲ 0 ▼



zii.hrs

First: Submit in PyPy instead of CPython

Second: Convert the input to a list of ints once instead of converting it in each iteration.

→ [Reply](#)

6 years ago, [hide](#) # | ☆

▲ 0 ▼



AbhishekAg

O(1) [solution](#) for problem D Segment Intersections.

→ [Reply](#)



AbhishekAg

6 years ago, [hide](#) # ^ | ☆

▲ 0 ▼

If anyone is interested in the explanation then comment.

→ [Reply](#)



ValarDohaeris_

6 years ago, [hide](#) # ^ | ☆

▲ 0 ▼

interested

→ [Reply](#)



6 years ago, [hide](#) # ^ | ☆

▲ +3 ▼

There can only be three initial configurations for the two intervals:



1> One completely overlaps the other.

2> There is some partial intersection b/w them.

3> They are disjoint.

Now, let us tackle them one by one.

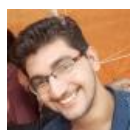
1>> We will have an initial I which can be calculated easily, thereafter if the current I is smaller than k we can increase the intersection length by one for all n pair of segments in just 1 step and we can do this untill all the pairs have become equal. It is obvious that we cannot increase I by more than 1 in a single operation, So this is optimal. Now if still I is less than k we only have one way to increase I by increasing the segment lengths of a pair this will give us a +1 for each 2 step. Also, we can do this indefinitely.

2>> Its the same as the first one except that the initial I has to be calculated differently.

3>> Initial $I=0$, Now how can we increase I ? we first have to make a pair of segment meet each other while doing this our I remains the same, lets call this cost as gap. Now as soon as the segments meet we can start having a bunch of +1s. Let us calculate how many +1s we can get in a row, Suppose the worst case when initially both segments are points, then we will have no of +1s equal to the no of +0s (we used to fill the gap). And in every other case the no of +1s will be greater than +0s, Remember this. Now let us see what choice do we have after using up these two operations, At this point we reach a situation analogous to one explained earlier in 1>> where we will be getting +1 for every 2 steps. We have all the tools ready. Let us expand this for n . I claim that in an optimal final solution we will have a situation where x of the n segments are completely overlapping and at most 1 pair of segments which has been extended by a 2 step operation or has a partial overlap, Also x will be maximum. Suppose a case where x is not the maximum then we have a case where $x-1$ segments have complete overlap and 1 segment with a partial overlap or an extended 2 step operation segment. We can simply rule out the first case because we can never compensate for the loss of one completely overlapping segment with a partial one. As for the other case the no of 2 step operation will be greater than the complete overlap of the segments as compensation. This is where we use the fact we proved earlier, that we can get a complete overlapping with total no of steps less than or equal to twice the size of the segments, which will always be at least as efficient as the +2 step operation and hence we can carve out a completely overlapping segment from this segment without losing optimality (for the lack of better word).

→ [Reply](#)

AbhishekAg



ValarDohaeris_

6 years ago, [hide](#) # [^](#) | [☆](#)

[+3](#)

Thanks, man for the effort! stating 3rd case more mathematically -

suppose we have to make 'rem' out of 'n' segs, let 'ex' be the amount to 'activate'



the seg $= \max(r1, l2) - \min(l1, r2)$, and tot
 $(= \max(r2, r1) - \min(l1, l2))$ we can get by
 doing 1 inc per 1 move operation (+1/1
 ops) after 'activation'.

Its oBvious that we have to 'activate' the
 1st seg and do +1/1 ops If still we have
 something remaining (let it be 'x')

now if 'x' $\geq$ 'tot' then its better to activate
 another seg and get tot why? because
 'tot'+ 'ex' $\leq$ 'tot'*2 as 'ex' $\leq$ 'tot'

if 'x' $<$ 'tot' then we decide on the basis of
 which is bigger, 'x'+ 'ex' or 'x'*2

so how is this $O(1)$ -> after the 1st
 activation, we can activate segs until reqd
 amount $<$ 'tot' and then we decide
 between 'activation'+ '+1/1 ops' and '+1/2
 ops' for the last one.

→ Reply

6 years ago, [hide](#) # 0



AbhishekAg

Yeah at one point I thought the
 way I described it isn't going to
 help anyone, happy to see that
 you got it... P.S. yours seems
 much formal and clear.

→ Reply



not_a_coder26

6 years ago, [hide](#) #

88381812

→ Reply

← Rev. 3 -8

6 years ago, [hide](#) #

← Rev. 3 0

Try to write the code in the code snippet. Yours is very unreadable and
 takes a lot of space in the comments



sup_bro

```
for(int i = 0; i < n; i++) {
    likeThis();
}
```

EDIT: For your problem, a string is good if it has all same characters in
 case or even length, or same parity indexed characters are same in
 case of odd length.

→ Reply



sapphire

6 years ago, [hide](#) #

0

Can anyone provide the Test case 2 — 19th input for task B. Most people had
 this error "expected 218 found 216". If not exact same test case, then maybe
 something similar?

→ Reply



anurag122898

6 years ago, [hide](#) #

0

I don't know the exact test case but you should also handle for the case
 k=0 explicitly because my solution got accepted after I make a case for
 k=0.

→ Reply



6 years ago, [hide](#) #

0

in problem B, the tutorial assumes that the left steps will always be accompanied



Phoenix23

by right steps as they have been taken in pairs but it may be happen that the last step will be a left step, so in that case the tutorial fails and so does the solution in the following test case

18 11 4 11 19 18 19 19 5 14 15 17 4 10 9 8 17 9 2 15 10

→ [Reply](#)



AvsUmNVipeR

6 years ago, [hide](#) # [^](#) | [☆](#)

▲ 0 ▼

The tutorial assumes left steps are **always preceded** by a right step; Hence the order (right,left).

→ [Reply](#)



Phoenix23

6 years ago, [hide](#) # [^](#) | [☆](#)

▲ 0 ▼

oh thanks... i had misunderstood it

→ [Reply](#)

6 years ago, [hide](#) # | [☆](#)

← Rev. 2

▲ 0 ▼

The idea of G is quite similar to this problem [Museum Tour](#)

Very interesting:

Undirected Graph -> find Bridges -> Tree -> Can be solved using DP

Directed Graph -> find SCC -> DAG -> Can be solved using DP

→ [Reply](#)



Aesir



MrD3viL

6 years ago, [hide](#) # | [☆](#)

▲ 0 ▼

Not expecting DP in second problem.

→ [Reply](#)



seniLogik

6 years ago, [hide](#) # | [☆](#)

← Rev. 3

▲ 0 ▼

[88352400](#) for problem C , can someone please tell me what's wrong in my code?

Spoiler

→ [Reply](#)



sumit_gaurav

6 years ago, [hide](#) # | [☆](#)

▲ +8 ▼

I have tried to make editorial for questions A-E . please have a look. Language :- Hindi

https://www.youtube.com/watch?v=S_-BaaH7P80&list=PLrT256hfFv5X1GNpiqEh5njN_WJmQu8Q6

[BaaH7P80&list=PLrT256hfFv5X1GNpiqEh5njN_WJmQu8Q6](#)

→ [Reply](#)



shubhanshushekharp

6 years ago, [hide](#) # [^](#) | [☆](#)

▲ +6 ▼

Great Tutorial!!

→ [Reply](#)



Pulkit_07

5 years ago, [hide](#) # | [☆](#)

▲ 0 ▼

In problem C, if we are using brute force, for all 100 combinations it is $O(n)$ then it becomes overall $O(100 \cdot n)$ which is $O(10^7)$ and with that we have 1000 test cases also, so how does it even work? At first i didn't do it because i thought maybe it will cause TLE, but I realized that they have done the same thing.

→ [Reply](#)





Zaid_Akhter

4 years ago, [hide](#) <#> | ☆

▲ 0 ▼

<https://codeforces.com/contest/1389/submission/162863152> what is wrong in my code, pls help→ [Reply](#)3 years ago, [hide](#) <#> | ☆

▲ 0 ▼

Brainless segment tree solution for problem F:

1. Coordinate compress the left and right bounds of all the segments, so that we can build a segment tree on these segments.
2. I'll just call the two types of segments as "Red" and "Blue". Create two segment trees, one for the red segments R and one for the blue segments B .
3. i 'th Leaf Node of segment tree R will hold two values: dp_i which is the maximum number of segments we can choose if the last segment chosen is red (till that iteration, context to be given later) and cnt_i which will store number of blue segments with $leftbound > i$ (till that iteration). Segment tree B is also symmetric to R , just interchange blue and red segments in definition. These segment trees will allow us to query $max(cnt_i + dp_i)$ in some range.
4. Sort all segments in increasing order of $rightbound$ and iterate over them. Let's say you are at red node, then you will transition from segment tree B . So firstly increment cnt for all leaf nodes in B in range $(1, leftbound - 1)$, as red and blue segments cant intersect. To transition, you just have to do a max query on range. (Once again, for blue segments, everything is symmetric, you will perform shit o R in that case.)

Implementation: [link](#)→ [Reply](#)

god_thor001

new, 4 months ago, [hide](#) <#> | ☆

▲ 0 ▼

dp solution for B (<https://codeforces.com/contest/1389/submission/368151676>)→ [Reply](#)

Supported by

